

Practice Problems on Memory Virtualization

1. Briefly state four distinct reasons (or advantages) why modern OS support/implement memory virtualization
2. OS requires certain support from the H/W in order to effectively implement “Limited Direct Execution” mechanism for CPU and memory virtualization. Briefly state any two distinct support/aides required from the H/W and its purpose/reason for each of the following cases:
 - a) Specific to CPU virtualization mechanism
 - b) Specific to Memory virtualization mechanism
 - c) Common to both CPU and Memory virtualization mechanism
3. An early OS used segmentation to provide some form of primitive virtual memory. The system only had 2 segments (segment 0 for code and a growing heap, segment 1 for a negatively-growing stack). The virtual address space size was only 128 bytes, and there was only 1K of physical memory. Given below is a set of traces from one old program that tells you which virtual address (VA) was accessed (in both hex and decimal forms), and then whether or not the access was valid or not (and hence a segmentation violation). If valid, the physical address (in both hex and decimal) are reported. Oddly enough, programs in this OS were allowed to keep running after memory-access violations, and thus we have a long trace that continues even after such a violation occurred. Here is the trace:

```
VA 0x0000006c (decimal: 108) --> VALID in SEG1: 0x000003ec (decimal: 1004)
VA 0x0000001d (decimal: 29) --> VALID in SEG0: 0x0000021d (decimal: 541)
VA 0x00000050 (decimal: 80) --> SEGMENTATION VIOLATION (SEG1)
VA 0x0000001e (decimal: 30) --> SEGMENTATION VIOLATION (SEG0)
VA 0x00000058 (decimal: 88) --> VALID in SEG1: 0x000003d8 (decimal: 984)
VA 0x00000061 (decimal: 97) --> VALID in SEG1: 0x000003e1 (decimal: 993)
VA 0x00000035 (decimal: 53) --> SEGMENTATION VIOLATION (SEG0)
VA 0x00000021 (decimal: 33) --> SEGMENTATION VIOLATION (SEG0)
VA 0x00000064 (decimal: 100) --> VALID in SEG1: 0x000003e4 (decimal: 996)
VA 0x0000003d (decimal: 61) --> SEGMENTATION VIOLATION (SEG0)
VA 0x0000000c (decimal: 12) --> VALID in SEG0: 0x0000020c (decimal: 524)
VA 0x00000005 (decimal: 5) --> VALID in SEG0: 0x00000205 (decimal: 517)
VA 0x0000002f (decimal: 47) --> SEGMENTATION VIOLATION (SEG0)
```

Now, let's try to recover some values. Note: In case it is not possible to perfectly recover any particular value; provide the best answer you can based on the information above. Briefly explain your answers. From this trace,

- a) What was the base and bounds register of segment 0 set to?
 - b) What was the base and bound registers of segment 1 set to?
4. One day Govind had an inspiration. He observes that most programs have majority of their code/data at the beginning of the address space. For his homegrown custom OS, Govind decides that he is going to implement paging mechanism using a two-level page table structure with the following twist: The first half of the page table entries in the first level page table (Page Directory) directly maps physical pages, and the second half of the entries map to the second level page tables as normal. Call the first half entries as fast, and the second half as normal. For the following questions, assume that the page size is 4 KB, size of PTE is 4 bytes and that all the page tables fit into a single page.
- (a) How many virtual pages are fast pages?
 - (b) How many virtual pages are normal pages?
 - (c) What is the maximum size of an address space in bytes (use exponential notation for convenience, e.g., 2^{23})?
- If TLB fetch time is T (with hit ratio as H) and Memory access time is M , what is the time taken for a memory reference (assume all referenced pages in memory) whose entry is in:
- (d) fast page
 - (e) normal page?
5. Consider a system with paging-based memory management, whose architecture allows for a 4GB virtual address space for processes. The size of logical pages and physical frames is 4KB. The system has 8GB of physical RAM. The system allows a maximum of 1K ($=1024$) processes to run concurrently. Assuming the OS uses hierarchical paging, calculate the maximum memory space required to store the page tables of *all* processes in the system. Assume that each page table entry requires an additional 10 bits (beyond the frame number) to store various flags. Assume page table entries are rounded up to the nearest byte. Consider the memory required for both outer and inner page tables in your calculations.
6. Consider a simple system running a single process. The size of physical frames and logical pages is 16 bytes. The RAM can hold 3 physical frames. The virtual addresses of the process are 6 bits in size. The program generates the following 20 virtual address references as it runs on the CPU: 0, 1, 20, 2, 20, 21, 32, 31, 0, 60, 0, 0, 16, 1, 17, 18, 32, 31, 0, 61. (Note: the 6-bit addresses are shown in decimal here.) Assume that the physical frames in RAM are initially empty and do not map to any

logical page.

- (a) Translate the virtual addresses above to logical page numbers referenced by the process. That is, write down the reference string of 20 page numbers corresponding to the virtual address accesses above. Assume pages are numbered starting from 0, 1, ...
 - (b) Calculate the number of page faults generated by the accesses above, assuming a FIFO page replacement algorithm. You must also correctly point out which page accesses in the reference string shown by you in part (a) are responsible for the page faults.
 - (c) Repeat (b) above for the LRU page replacement algorithm.
 - (d) What would be the lowest number of page faults achievable in this example, assuming an optimal page replacement algorithm were to be used? Repeat (b) above for the optimal algorithm.
7. Consider a system with only virtual addresses, but no concept of virtual memory or demand paging. Define *total memory access time* as the time to access code/data from an address in physical memory, including the time to resolve the address (via the TLB or page tables) and the actual physical memory access itself. When a virtual address is resolved by the TLB, experiments on a machine have empirically observed the total memory access time to be (an approximately constant value of) t_h . Similarly, when the virtual address is not in the TLB, the total memory access time is observed to be t_m . If the average total memory access time of the system (averaged across all memory accesses, including TLB hits as well as misses) is observed to be t_x , calculate what fraction of memory addresses are resolved by the TLB. In other words, derive an expression for the TLB hit rate in terms of t_h , t_m , and t_x . You may assume $t_m > t_h$.
8. A system has given specifications: its virtual address is 32 bits long, its page size is 4KB, its physical memory has 1024 page frames and its TLB can store 32 page entries. In this problem, we will try to understand what happens while running some test code. Before running the test code, though, we first run the following *initialization code*.

```
int k = 0;

void *store = malloc(NUM_PAGES * PAGE_SIZE);

void *ptr = store;

while(k++ < NUM_PAGES) {
    (int *) *ptr = 0; // initialize first value on each page
    ptr += PAGE_SIZE;
}
```

The initialization code allocates some amount of memory and sets the first integer of each page to 0. Assuming malloc() returns page-aligned memory in this example. After running the initialization code, we run the following *test code*:

```

ptr = store;
k=0;
while(k++ < NUM_PAGES) {
    int x = (int *) *ptr; // load value pointed to by ptr
    ptr += PAGE_SIZE;
}

```

We are only interested in memory accesses to the malloc'd region through ptr and ignore all other memory accesses (i.e., ignore instruction fetches and stores to x). We further assume that memory and TLB start from a clean state before we run the initialization code. Given that LRU replacement policy is used whenever such a policy might be needed by either hardware or software: Fill the table with no. of TLB hits, misses and page faults that occur during the test code when:

	TLB hits	TLB misses	Page Faults
(a) NUM PAGES is 32?			
(b) NUM PAGES is 1024?			
(c) NUM PAGES is 2048?			

9. Consider a system with a 6 bit virtual address space, and 16 byte pages/frames. The mapping from virtual page numbers to physical frame numbers of a process is (0,8), (1,3), (2,11), and (3,1). Translate the following virtual addresses to physical addresses. Note that all addresses are in decimal. You may write your answer in decimal or binary.
 - (a) 20
 - (b) 40
10. Consider a system with several running processes. The system is running a modern OS that uses virtual addresses and demand paging. It has been empirically observed that the memory accesstimes in the system under various conditions are: t1 when the logical memory address is foundin TLB cache, t2 when the address is not in TLB but does not cause a page fault, and t3 when the address results in a page fault. This memory access time includes all overheads like page faultservicing and logical-to-physical address translation. It has been observed that, on an average,10% of the logical address accesses result in a page fault. Further, of the remaining virtual address accesses, two-thirds of them can be translated using the TLB cache, while one-third require walking the page tables. Using the information provided above, calculate the average expected memory access time in the system in terms of t1,t2, and t3.
11. Consider a system where each process has a virtual address space of 2^v bytes. The physical address space of the system is 2^p bytes, and the page size is 2^k bytes. The size of each page table entry is 2^e bytes. The system uses hierarchical paging with l levels of page tables, where the page table entries in the last level point to the actual physical pages of the

$\geq$

process. Assume/ 2. Let v_0 denote the number of (most significant) bits of the virtual address that are used as an index into the outermost page table during address translation.

- (a) What is the number of logical pages of a process?
 - (b) What is the number of physical frames in the system?
 - (c) What is the number of PTEs that can be stored in a page?
 - (d) How many pages are required to store the innermost PTEs?
 - (e) Derive an expression for l in terms of v , p , k , and e .
 - (f) Derive an expression for v_0 in terms of l , v , p , k , and e .
12. Consider an operating system that uses 48-bit virtual addresses and 16KB pages. The system uses a hierarchical page table design to store all the page table entries of a process, and each page table entry is 4 bytes in size. What is the total number of pages that are required to store the page table entries of a process, across all levels of the hierarchical page table?
13. Consider a system with 8-bit virtual and physical addresses, and 16 byte pages. A process in this system has 4 logical pages, which are mapped to 3 physical pages in the following manner: logical page 0 maps to physical page 6, 1 maps to 3, 2 maps to 11, and logical page 5 is not mapped to any physical page yet. All the other pages in the virtual address space of the process are marked invalid in the page table. The MMU is given a pointer to this page table for address translation. Further, the MMU has a small TLB cache that stores two entries, for logical pages 0 and 2. For each virtual address shown below, describe what happens when that address is accessed by the CPU. Specifically, you must answer what happens at the TLB (hit or miss?), MMU (which page table entry is accessed?), OS (is there a trap of any kind?), and the physical memory (which physical address is accessed?). You may write the translated physical address in binary format. (Note that it is not implied that the accesses below happen one after the other; you must solve each part of the question independently using the information provided above.)
- (a) Virtual address 7
 - (b) Virtual address 20
 - (c) Virtual address 70
 - (d) Virtual address 80
14. Consider a system with 8-bit addresses and 16-byte pages. A process in this system has 4 logical pages, which are mapped to 3 physical frames in the following manner: logical page 0 maps to physical frame 2, page 1 maps to frame 0, page 2 maps to frame 1, and page 3 is not mapped to any physical frame. The process may not use more than 3 physical frames. On a page fault, the demand paging system uses the LRU policy to evict a page. The MMU has a TLB cache that can store 2 entries. The TLB cache also uses the LRU policy to store the most recently used mappings in cache. Now, the process accesses the

following logical addresses in order: 7, 17, 37, 20, 40, 60.

- (a) Out of the 6 memory accesses, how many result in a TLB miss? Clearly indicate the accesses that result in a miss. Assume that the TLB cache is empty before the accesses begin.
 - (b) Out of the 6 memory accesses, how many result in a page fault? Clearly indicate the accesses that result in a page fault.
 - (c) Upon accessing the logical address 60, which physical address is eventually accessed by the system (after servicing any page faults that may arise)? Show suitable calculations.
15. Consider a 64-bit system running an OS that uses hierarchical page tables to manage virtual memory. Assume that logical and physical pages are of size 4KB and each page table entry is 4 bytes in size.
- (a) What is the maximum number of levels in the page table of a process, including both the outermost page directory and the innermost page tables?
 - (b) Indicate which bits of the virtual address are used to index into each of the levels of the page table.
 - (c) Calculate the maximum number of pages that may be required to store all the page table entries of a process across all levels of the page table.
16. The page size in a system (running a Linux-like operating system on x86 hardware) is increased while keeping everything else (including the total size of main memory) the same. For each of the following metrics below, indicate whether the metric is *generally* expected to increase, decrease, or not change as a result of this increase in page size.
- (a) Size of the page table of a process
 - (b) TLB hit rate
 - (c) Internal fragmentation of main memory
17. Consider a process with 4 logical pages, numbered 0–3. The page table of the process consists of the following logical page number to physical frame number mappings: (0, 11), (1, 35), (2, 3), (3, 1). The process runs on a system with 16 bit virtual addresses and a page size of 256 bytes. You are given that this process accesses virtual address 770. Answer the following questions, showing suitable calculations.
- (a) Which logical page number does this virtual address correspond to?
 - (b) Which physical address does this virtual address translate to?

18. Consider a system with 16 bit virtual addresses, 256 byte pages, and 4 byte page table entries. The OS builds a multi-level page table for each process. Calculate the maximum number of pages required to store all levels of the page table of a process in this system.
19. Consider a process with 4 physical pages numbered 0–3. The process accesses pages in the following sequence: 0, 1, 0, 2, 3, 3, 0, 2. Assume that the RAM can hold only 3 out of these 4 pages, is initially empty, and there is no other process executing on the system.
 - (a) Assuming the demand paging system is using an LRU replacement policy, how many page faults do the 8 page accesses above generate? Indicate the accesses which cause the faults.
 - (b) What is the minimum number of page faults that would be generated by an optimal page replacement policy? Indicate the accesses which cause the faults.
20. Consider a Linux-like operating system running on a 48-bit CPU hardware. The OS uses hierarchical paging, with 8 KB pages and 4 byte page table entries.
 - (a) What is the maximum number of levels in the page table of a process, including both the outermost page directory and the innermost page tables?
 - (b) Indicate which bits of the virtual address are used to index into each of the levels of the page table.
 - (c) Calculate the maximum number of pages that may be required to store all the page table entries of a process across all levels of the page table.
21. Consider the scenario described in the previous question. You are told that the OS uses demand paging. That is, the OS allocates a physical frame and a corresponding PTE in the page table only when the memory location is accessed for the first time by a process. Further, the pages at all levels of the hierarchical page table are also allocated on demand, i.e., when there is at least one valid PTE within that page. A process in this system has accessed memory locations in 4K unique pages so far. You may assume that none of these 4K pages has been swapped out yet. You are required to compute the minimum and maximum possible sizes of the page table of this process after all accesses have completed.
 - (a) What is the minimum possible size (in pages) of the page table of this process?
 - (b) What is the maximum possible size (in pages) of the page table of this process?
22. In a demand paging system, it is intuitively expected that increasing the number of physical frames will naturally lead to a reduction in the rate of page faults. However, this intuition does not hold for some page replacement policies. A replacement policy is said to suffer from *Belady's anomaly* if increasing the number of physical frames in the system can sometimes lead to an increase in the number of page faults. Consider two page replacement policies studied in class: FIFO and LRU. For each of these two policies, you must state if the policy can suffer from Belady's anomaly (yes/no). Further, if you answer yes, you must provide an example of the occurrence of the anomaly, where increasing the number of physical frames actually lead to an increase in the number of page faults. If you

answer no, you must provide an explanation of why you think the anomaly can never occur with this policy.

Hint: you may consider the following example. A process has 5 logical pages, and accesses them in this order: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5. You may find this scenario useful in finding an example of Belady's anomaly. Of course, you may use any other example as well.

(a) FIFO

(b) LRU